

Лабораторна робота № 12

Мета: Ознайомлення з поведінковими патернами проектування, практичне застосування патернів та їх реалізація в обраній мові програмування.

Завдання:

ЗАВДАННЯ 1

Відповідно до варіанту (додаток А, патерн 1) та предметної області (додаток В) реалізувати приклад застосування поведінкового патерну проектування на обраній мові програмування в консольному додатку.

ЗАВДАННЯ 2 Базове розгортання проекту на віддаленому сервері

Відповідно до варіанту (додаток А, патерн 2) та предметної області (додаток В) реалізувати приклад застосування поведінкового патерну проектування на обраній мові програмування в консольному додатку.

ЗАВДАННЯ 3

Відповідно до варіанту (додаток А, патерн 3) та предметної області (додаток В) додати до попередньої реалізації WebApi з ЛР4-6 застосування поведінкового патерну проектування.

Хід роботи:

Завдання 1

Реалізовано виконання завдання із використанням поведінкового патерну Chain of Responsibility:

Створено модель користувача:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
namespace Task1.Models
{
    public class User
    {
        public List<Sub> subs { get; private set; }
        public string Name { get; set; }
        public double Balance { get; set; }
    }
}
```

					ЛР.ОК24.ПІ231.03.12		
Змін.	Аркуш	№ докум.	Підпис	Дата	Behavioral Design Patterns		
Розробив	Башук О. Ю.						
Перевірів	Жереб Д. В.						
Н.контр.							
Затвер.							
					Літ	Аркуш	Аркушів
					н	1	16
					Х П К		

```

public bool IsBlocked { get; set; }
public User(string name, double balance, bool isBlocked)
{
    Name = name;
    Balance = balance;
    IsBlocked = isBlocked;
    subs = new List<Sub>();
}
public void AddSub(Sub sub) =>
    subs.Add(sub);
public void RemoveSub(Sub sub)
{
    if (!subs.Contains(sub))
        throw new ArgumentException();
    subs.Remove(sub);
}
}
}

```

Створено модель підписки:

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
namespace Task1.Models
{
    public class Sub
    {
        public string Service { get; set; }
        public string Type { get; set; }
        public double Cost { get; set; }

        public Sub(string service, string type, double cost)
        {
            Service = service;
            Type = type;
            Cost = cost;
        }
    }
}

```

Створено абстрактний клас для реалізації потоку обробки, під час використання його як батьківського:

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using Task1.Models;
namespace Task1.Abstract
{
    public abstract class SubHandler

```

					ЛР.ОК24.ПІ231.03.12	Арк.
						2
Вим.	Арк.	№ докум.	Підпис	Дата		

```

    {
        protected SubHandler Next;
        public SubHandler SetNext(SubHandler next)
        {
            Next = next;
            return next;
        }
        public abstract void Handle(User user, Sub sub);
    }
}

```

Створено клас BalanceHandler, що наслідує абстрактний клас:

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using Task1.Abstract;
using Task1.Models;
namespace Task1.Handlers
{
    public class BalanceHandler : SubHandler
    {
        public override void Handle(User user, Sub sub)
        {
            if(user.Balance < sub.Cost)
            {
                Console.WriteLine("Недостатньо коштів");
                return;
            }
            Console.WriteLine("Баланс відповідає потребам");
            if(Next is not null)
                Next.Handle(user, sub);
        }
    }
}

```

Створено клас UserBlockHandler, що наслідує абстрактний клас:

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using Task1.Abstract;
using Task1.Models;
namespace Task1.Handlers
{
    public class UserBlockHandler : SubHandler
    {
        public override void Handle(User user, Sub sub)
        {
            if(user.IsBlocked)
            {

```

					ЛР.ОК24.ПІ231.03.12	Арк.
						3
Вим.	Арк.	№ докум.	Підпис	Дата		

```

        Console.WriteLine("Користувач заблокований");
        return;
    }
    Console.WriteLine("Користувач не заблокований");
    if(Next is not null)
        Next.Handle(user, sub);
    }
}

```

Створено клас SubTypeHandler, що наслідуює абстрактний клас:

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using Task1.Abstract;
using Task1.Models;
namespace Task1.Handlers
{
    public class SubTypeHandler : SubHandler
    {
        public override void Handle(User user, Sub sub)
        {
            if(sub.Type != "Standart" || sub.Type == "Premium")
            {
                Console.WriteLine($"Невірний тип підписки: {sub.Type}");
                return;
            }
            Console.WriteLine("Вірний тип підписки");
            if(Next is not null)
                Next.Handle(user, sub);
        }
    }
}

```

Створено фінальний клас в ланцюгу викликів, який також наслідуює абстрактний клас:

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using Task1.Abstract;
using Task1.Models;
namespace Task1.Handlers
{
    public class FinalSteepHandler : SubHandler
    {
        public override void Handle(User user, Sub sub)
        {
            user.Balance -= sub.Cost;
            user.AddSub(sub);
        }
    }
}

```

					ЛР.ОК24.ПІ231.03.12	Арк.
						4
Вим.	Арк.	№ докум.	Підпис	Дата		

```

        Console.WriteLine($"Підписка оформлена для користувача: {user.Name}");
    }
}
}

    Організовано головну програму:
using System.Text;
using Task1.Abstract;
using Task1.Handlers;
using Task1.Models;
public class Program
{
    static void Main(string[] args)
    {
        Console.OutputEncoding = Encoding.UTF8;
        User user = new User("User",1000,false);
        Sub sub = new Sub("Netflix","Standart",500);
        SubHandler handler = new UserBlockHandler();
        handler.SetNext(new BalanceHandler())
            .SetNext(new SubTypeHandler())
            .SetNext(new FinalSteepHandler());
        handler.Handle(user, sub);
        Console.WriteLine($"Результат додавання підписки: {user.subs[0].Service} присутня"
    }
}

```

Протестовано роботу програми:

```

Користувач не заблокований
Баланс відповідає потребам
Вірний тип підписки
Підписка оформлена для користувача: User
Результат додавання підписки: Netflix присутня

```

Рисунок 1 – Результат виконання програми

Завдання 2

Рналізовано виконання завдання за допомогою використання паттерну – посередник:

Створено інтерфейс для посередника:

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using Task2.Models;
namespace Task2.Interfaces
{
    public interface ISubMeddiator
    {
        public void Subscribe(User user, Sub sub);
    }
}

```

					ЛР.ОК24.ПІ231.03.12	Арк.
						5
Вим.	Арк.	№ докум.	Підпис	Дата		

Створено клас – посередник, що реалізує вказаний інтерфейс:

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using Task2.Interfaces;
using Task2.Models;
using Task2.Parts;
namespace Task2
{
    public class SubMediator : ISubMediator
    {
        private Payment _paymentService;
        private Notification _notification;
        public SubMediator(Payment payment, Notification notification)
        {
            _paymentService = payment;
            _notification = notification;
        }
        public void Subscribe(User user, Sub sub)
        {
            Logger.Instance.Log($"Користувач {user.Name} хоче оформити підписку {sub.Name}");
            if (!_paymentService.Pay(user, sub))
            {
                _notification.Send(user, "не достатньо коштів для оформлення підписки");
                Logger.Instance.Log("Помилка, недостатньо коштів");
                return;
            }
            user.subs.Add(sub);
            _notification.Send(user, "підписку оформлено");
            Logger.Instance.Log("Успіх, підписка оформлена");
        }
    }
}

```

Створено частини, які будуть взаємодіяти за допомогою посередника:

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using Task2.Models;
namespace Task2.Parts
{
    public class Notification
    {
        public void Send(User user, string message)
        {
            Console.WriteLine($"Повідомлення до {user.Name}: {message}");
        }
    }
}

```

					ЛР.ОК24.ПІ231.03.12	Арк.
						6
Вим.	Арк.	№ докум.	Підпис	Дата		

```

    }
}
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using Task2.Models;
namespace Task2.Parts
{
    public class Payment
    {
        public bool Pay(User user, Sub sub)
        {
            if(user.Balance < sub.Price)
                return false;
            user.Balance -= sub.Price;
            return true;
        }
    }
}

```

Створено моделі об'єктів, які беруть участь у прямій роботі програми:

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
namespace Task2.Models
{
    public class User
    {
        public string Name { get; set; }
        public double Balance { get; set; }
        public List<Sub> subs { get; } = new List<Sub>();
        public User(string name, double balance)
        {
            Name = name;
            Balance = balance;
        }
    }
}
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
namespace Task2.Models
{
    public class Sub
    {
        public string Name { get; set; }
    }
}

```

					ЛР.ОК24.ПІ231.03.12	Арк.
						7
Вим.	Арк.	№ докум.	Підпис	Дата		

```

        public double Price { get; set; }
        public Sub(string name, double price)
        {
            Name = name; Price = price;
        }
    }
}

```

Розроблено головну програму:

```

using System.Text;
using Task2;
using Task2.Interfaces;
using Task2.Models;
using Task2.Parts;
public class Program
{
    static void Main(string[] args)
    {
        Console.OutputEncoding = Encoding.UTF8;
        User user = new User("User1", 1000);
        Sub sub = new Sub("Netflix", 500);
        ISubMeddiator subMeddiator = new SubMediator(new Payment(), new Notification());
        subMeddiator.Subscribe(user, sub);
    }
}

```

Протестовано роботу головної програми:

Повідомлення до User1: підписку оформлено

Рисунок 2 – Результат виконання програми

Завдання 3

Створено інтерфейс `IPeopleCommand` із двома методами `ExecuteAsync` – виконати команду та `UndoAsync` – повертає попередній стан БД та створено класи – команди, що реалізують цей інтерфейс та використовують в якості отримувача сервісу `PeopleService`: `namespace LW4_task_3.Command.Interface`

```

{
    public interface IPeopleCommand
    {
        Task ExecuteAsync();
        Task Undo();
    }
}
using AutoMapper;
using LW4_task_3.Command.Interface;
using LW4_task_3.Interface.Interfaces;
using LW4_task_3.Models.Entities;
using LW4_task_3.Models.Request;
using System.Windows.Input;
namespace LW4_task_3.Command.Commands
{
    public class CreateCommand : IPeopleCommand
    {

```

					ЛР.ОК24.ПІ231.03.12	Арк.
						8
Вим.	Арк.	№ докум.	Підпис	Дата		


```

private readonly IPeopleService _peopleService;
private readonly PeopleItem _people;
public CreateCommand(IPeopleService peopleService, PeopleItem people)
{
    _peopleService = peopleService;
    _people = people;
}
public async Task ExecuteAsync() =>
    await _peopleService.CreateAsync(_people);
public async Task Undo() =>
    await _peopleService.DeleteAsync(_people.Id);
}
}
using LW4_task_3.Command.Interface;
using LW4_task_3.Interface.Interfaces;
using LW4_task_3.Models.Entities;
namespace LW4_task_3.Command.Commands
{
    public class DeleteCommand : IPeopleCommand
    {
        private readonly IPeopleService _service;
        private readonly string _id;
        private PeopleItem _peopleItem;
        public DeleteCommand(IPeopleService service, string id)
        {
            _service = service;
            _id = id;
        }
        public async Task ExecuteAsync()
        {
            _peopleItem = await _service.GetByIdAsync(_id);
            await _service.DeleteAsync(_id);
        }
        public async Task Undo() =>
            await _service.CreateAsync(_peopleItem);
    }
}
using LW4_task_3.Command.Interface;
using LW4_task_3.Interface.Interfaces;
using LW4_task_3.Models.Entities;
namespace LW4_task_3.Command.Commands
{
    public class UpdateCommand : IPeopleCommand
    {
        private readonly IPeopleService _service;
        private readonly PeopleItem _updatedPeople;
        private readonly string _id;
        private PeopleItem _startPeople;
        public UpdateCommand(IPeopleService service, PeopleItem updatesPeople, string id)
        {
            _service = service;

```

					ЛР.ОК24.ПІ231.03.12	Арк.
						9
Вим.	Арк.	№ докум.	Підпис	Дата		

```

        _updatedPeople = updatesPeople;
        _id = id;
    }
    public async Task ExecuteAsync()
    {
        _startPeople = await _service.GetByIdAsync(_id);

        await _service.UpdateAsync(_id, _updatedPeople);
    }
    public async Task Undo() =>
        await _service.UpdateAsync(_id, _startPeople);
    }
}

```

Створено клас – виконувач та зареєстровано його в Program.cs:

```

using LW4_task_3.Command.Interface;
namespace LW4_task_3.Command.Invoker
{
    public class PeopleInvoker
    {
        private Stack<IPeopleCommand> _commands = new Stack<IPeopleCommand>();
        public async Task ExecuteAsync(IPeopleCommand command)
        {
            await command.ExecuteAsync();

            _commands.Push(command);
        }
        public async Task UndoAsync()
        {
            if (_commands.Count > 0)
            {
                var c = _commands.Pop();

                await c.Undo();
            }
        }
    }
}

```

Створено контролер для виконання запитів:

```

using AutoMapper;
using LW4_task_3.Interface.Interfaces;
using Microsoft.AspNetCore.Mvc;
using LW4_task_3.Command.Invoker;
using LW4_task_3.Models.Request;
using LW4_task_3.Command.Commands;
using LW4_task_3.Models.Entities;
using System.Text.Json;
namespace LW4_task_3.Command.Controller
{
    [Route("[controller]")]
    [ApiController]
    public class CommandController : ControllerBase
    {

```

					ЛР.ОК24.ПІ231.03.12	Арк. 10
Вим.	Арк.	№ докум.	Підпис	Дата		

```

{
    private readonly IPeopleService _peopleService;
    private readonly IMapper _mapper;
    private readonly PeopleInvoker _invoker;

    public CommandController(IPeopleService peopleService, IMapper mapper, PeopleInvoker invoker)
    {
        _peopleService = peopleService;
        _mapper = mapper;
        _invoker = invoker;
    }
    [HttpGet]
    public async Task<IActionResult> GetAll(string? name, string? email)
    {
        try
        {
            var peoples = await _peopleService.GetPeoplesItemsAsync(name, email);
            return Ok(peoples);
        }
        catch(KeyNotFoundException kex)
        {
            return NotFound(kex.Message);
        }
    }
    [HttpGet("{id}")]
    public async Task<IActionResult> GetById(string id)
    {
        try
        {
            var person = await _peopleService.GetByIdAsync(id);
            return Ok(person);
        }
        catch(KeyNotFoundException kex)
        {
            return NotFound(kex.Message);
        }
    }
    [HttpPost]
    public async Task<IActionResult> Create(PeopleRequest personRequest)
    {
        var person = _mapper.Map<PersonItem>(personRequest);
        await _invoker.ExecuteAsync(new CreateCommand(_peopleService, person));
        return CreatedAtAction(nameof(GetById), new { Id = person.Id }, person);
    }
    [HttpPut("{id}")]
    public async Task<IActionResult> Update(string id, PeopleRequest personRequest)
    {
        var person = _mapper.Map<PersonItem>(personRequest);
        try
        {

```

					ЛР.ОК24.ПІ231.03.12	Арк.
						11
Вим.	Арк.	№ докум.	Підпис	Дата		

```

        await _invoker.ExecuteAsync(new UpdateCommand(_peopleService, people, id));
        return NoContent();
    }
    catch(KeyNotFoundException kex)
    {
        return NotFound(kex.Message);
    }
}
[HttpPatch("{id}")]
public async Task<IActionResult> UpdatePart(string id, JsonElement jsonElement)
{
    try
    {
        var people = await _peopleService.GetByIdAsync(id);
        var updatePeople = _mapper.Map<PeopleItem>(people);
        if (jsonElement.TryGetProperty("name", out var name))
            updatePeople.Name = name.ToString();
        if(jsonElement.TryGetProperty("email", out var email))
            updatePeople.Email = email.ToString();
        if (!TryValidateModel(updatePeople))
            return BadRequest();

        await _invoker.ExecuteAsync(new UpdateCommand(_peopleService, updatePeople, id
    ));
        return NoContent();
    }
    catch(KeyNotFoundException kex)
    {
        return NotFound(kex.Message);
    }
}
[HttpDelete("{id}")]
public async Task<IActionResult> Delete(string id)
{
    try
    {
        await _invoker.ExecuteAsync(new DeleteCommand(_peopleService, id));
        return NoContent();
    }
    catch (KeyNotFoundException kex)
    {
        return NotFound(kex.Message);
    }
}
[HttpPost("undo")]
public async Task<IActionResult> Reset()
{
    try
    {
        await _invoker.UndoAsync();
        return Ok("Успішно");
    }
}

```

					ЛР.ОК24.ПІ231.03.12	Арк.
						12
Вим.	Арк.	№ докум.	Підпис	Дата		

```

    }
    catch(Exception ex)
    {
        return BadRequest(ex.Message);
    }
}
}
}

```

Протестовано роботу контролера із паттерном команда:



Рисунок 3 – Створено нового користувача

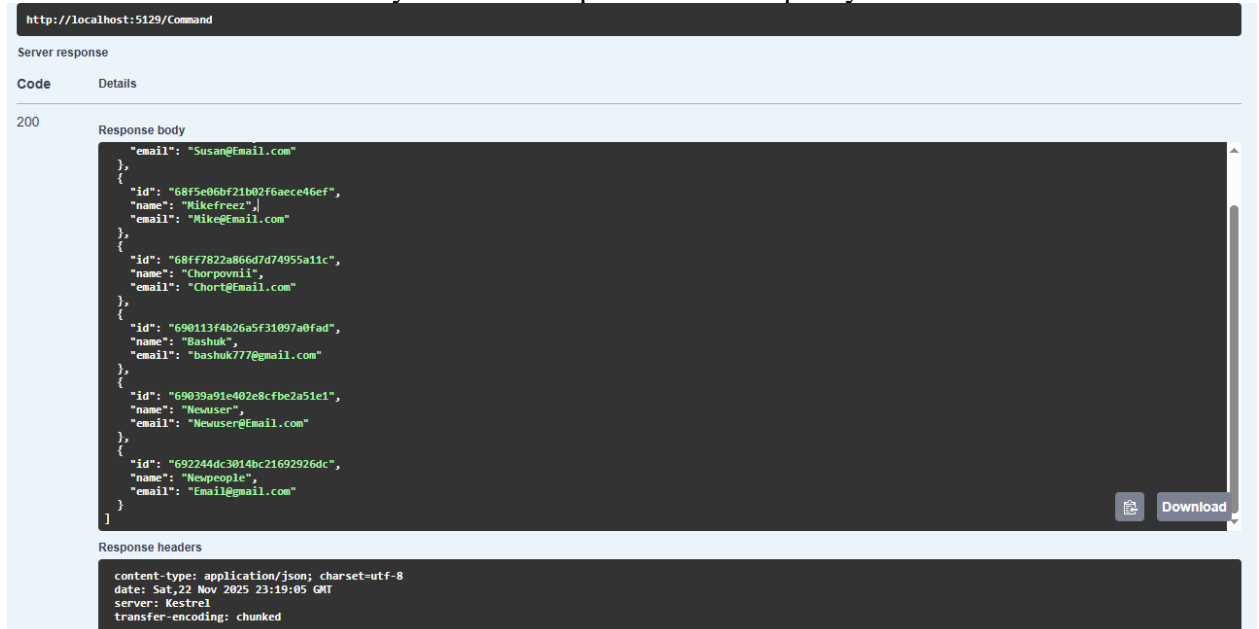


Рисунок 4 – Переглянуто список усіх користувачів



Рисунок 5 – Повернуто дію назад

					ЛР.ОК24.ПІ231.03.12	Арк.
						13
Вим.	Арк.	№ докум.	Підпис	Дата		

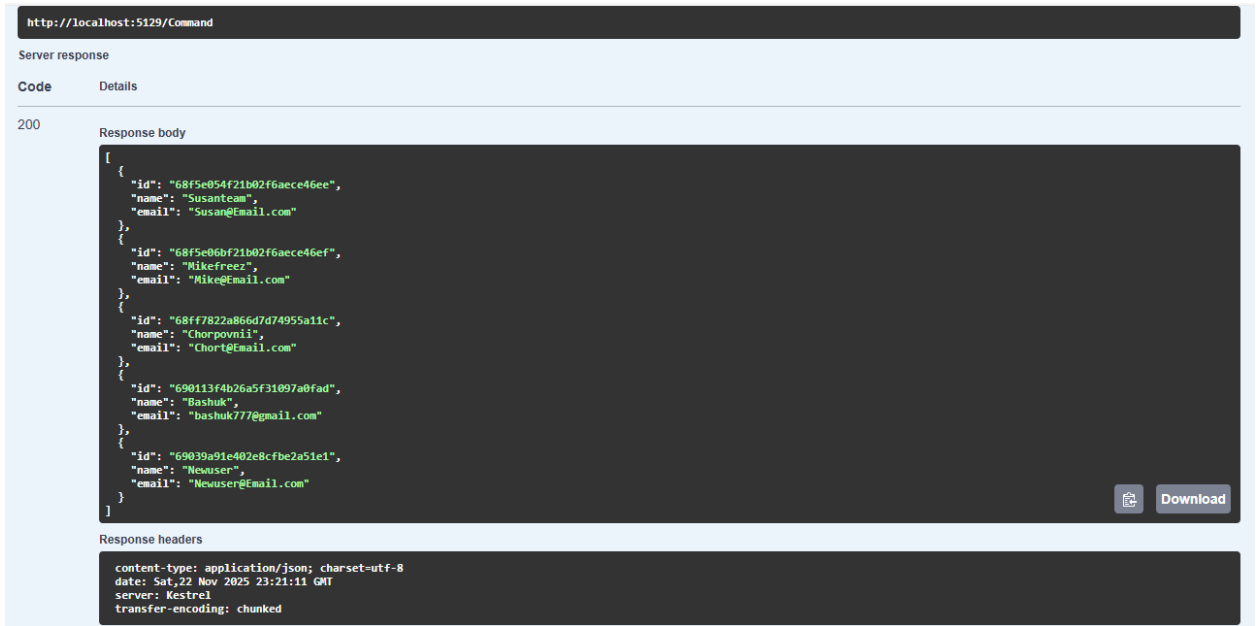


Рисунок 6 – Переглянуто список людей



Рисунок 7 – Оновлено поле в людині



Рисунок 8 – Отримано оновленого користувача



Рисунок 9 – Повернуто попереднє ім'я користувача



Рисунок 10 – Переглянуто результат повернення



Рисунок 11 – Видалено користувача

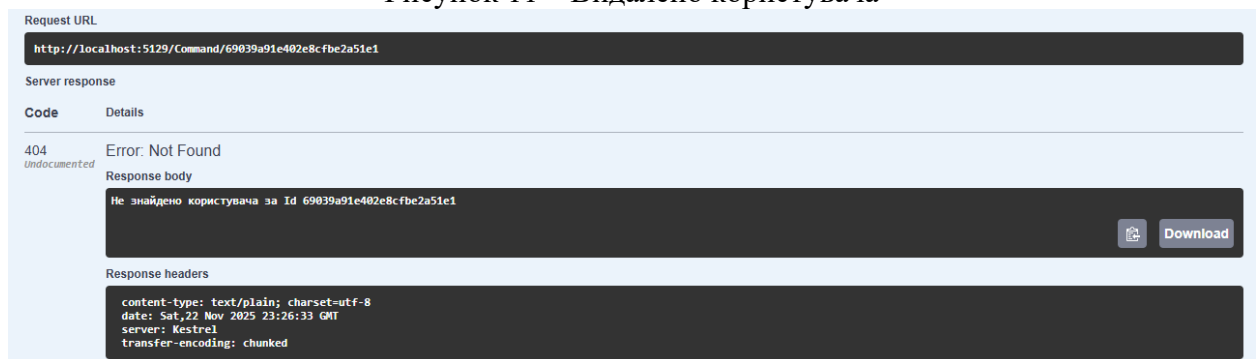


Рисунок 12 – Спроба отримати видаленого користувача

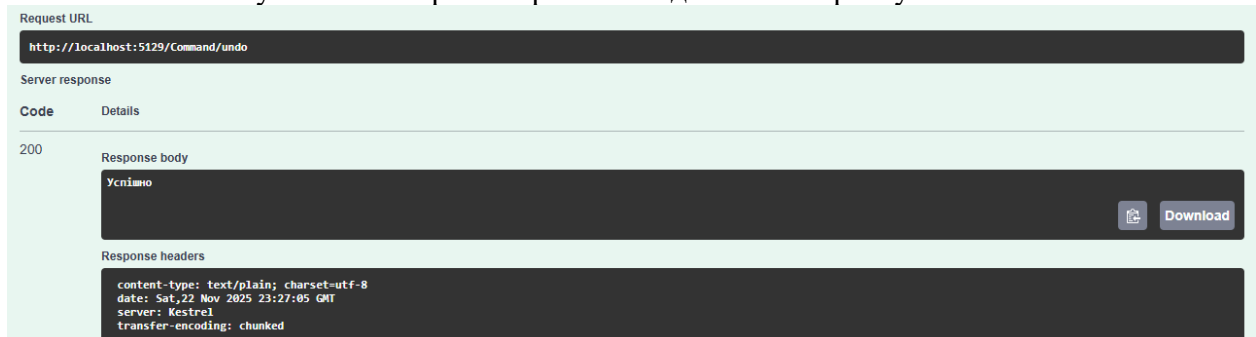


Рисунок 13 – Повернуто видаленого користувача

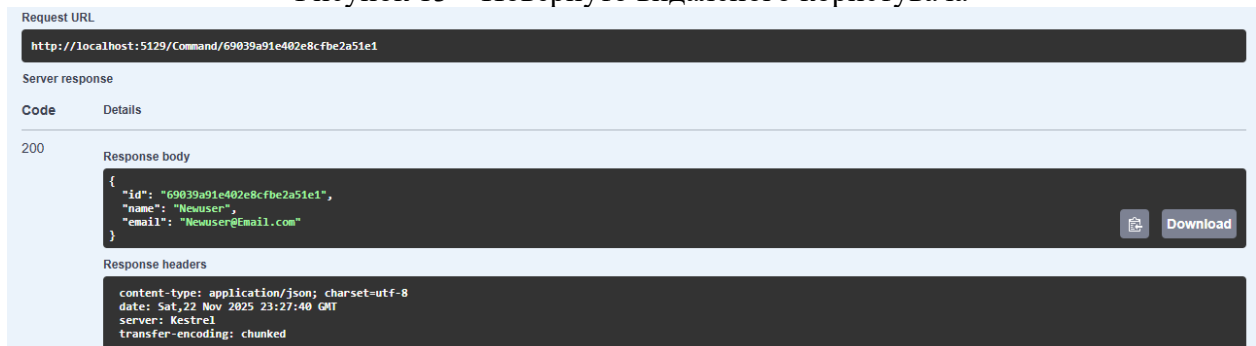


Рисунок 14 – Переглянуто повернутого користувача

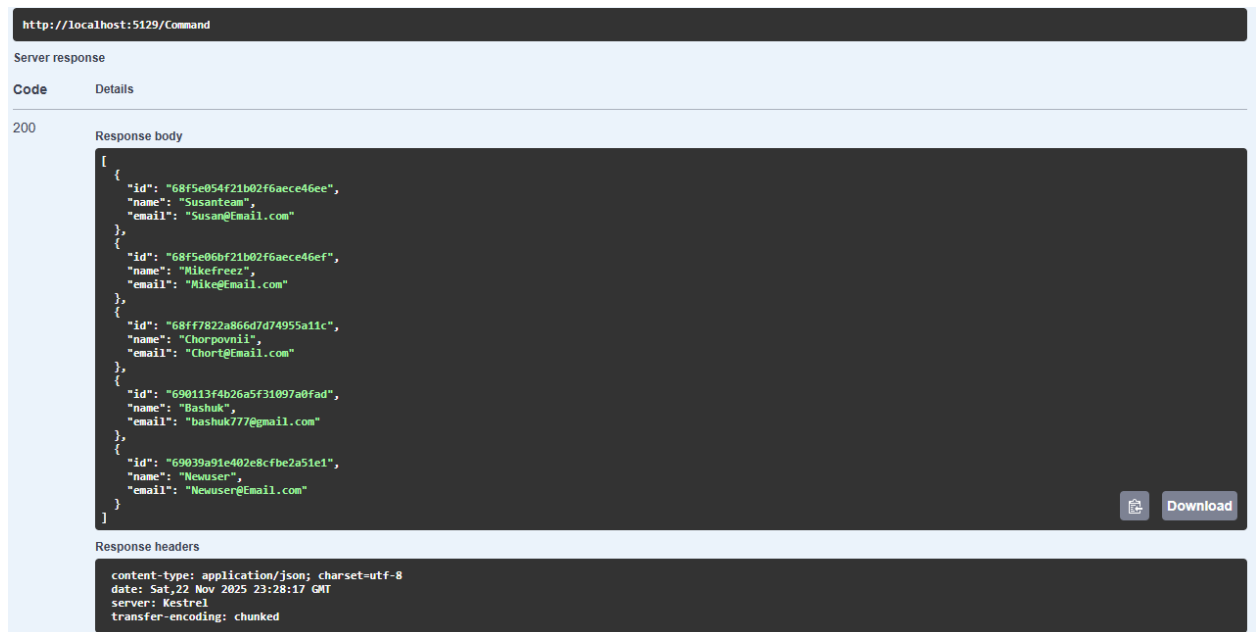


Рисунок 15 – Переглянуто список всіх користувачів

ВИСНОВОК:

На даній лабораторній вивчив основи роботи із поведінковими паттернами, отримав практичні нваички роботи із паттернами: Chain of Responsibility, Mediator, Command

					ЛР.ОК24.ПІ231.03.12	Арк.
						16
Вим.	Арк.	№ докум.	Підпис	Дата		